

CI/CD-Pipelines am Beispiel von GitLab Automatisches helm-Deployment by Example

Container mit Schwerpunkt
Kubernetes

- Zielsetzung
- Helm-Chart erstellen
- Helm-Chart testen
- Automatisches Deployment mit Helm

- Einsatz von Helm für das automatische Deployment.
- Deployment der Beispiel-Anwendung.
- Schritte der Entwicklung
 - Helm-Chart lokal erstellen.
 - Helm-Chart lokal testen.
 - Helm-Chart in das git-Repository übernehmen.
 - Build-Skript für das Helm-Deployment anpassen.

- Zielsetzung
- Helm-Chart erstellen
- Helm-Chart testen
- Automatisches Deployment mit Helm

Helm-Chart erstellen - Template-Struktur erzeugen

```
$ helm create containerapp
Creating containerapp
$ find .
.
./containerapp
./containerapp/templates
./containerapp/templates/serviceaccount.yaml
./containerapp/templates/hpa.yaml
./containerapp/templates/deployment.yaml
./containerapp/templates/service.yaml
./containerapp/templates/ingress.yaml
./containerapp/templates/_helpers.tpl
./containerapp/templates/httproute.yaml
./containerapp/templates/NOTES.txt
./containerapp/templates/tests
./containerapp/templates/tests/test-connection.yaml
./containerapp/charts
./containerapp/Chart.yaml
./containerapp/values.yaml
./containerapp/.helmignore
```

- *helm create*: erzeugt ein template für ein Helm-Chart 😊
- Benötigt werden folgende Templates:
 - deployment.yaml
 - service.yaml
 - ingress.yaml
 - übrige *.yaml-Dateien aus dem Ordner templates löschen.
- *values.yaml*: Enthält Parameter des Deployments.

Helm-Chart erstellen - Angaben zum Deployment

```
image:␣  
  · repository: ·<Name ·des ·Repositories>␣  
  · tag: ·latest␣  
  · pullPolicy: ·Always␣
```

```
imagePullSecrets:␣  
  · - name: ·gitlab-registry␣
```

- Anpassungen in values.yaml
- Das angegebene Secret muss im Namensraum bereits bekannt sein.

Helm-Chart erstellen - Angaben zum Service

```
ingress:␣  
  ··enabled: ·true␣  
  ··className: ·nginx␣  
  ··annotations:␣  
    ···nginx.ingress.kubernetes.io/use-regex: ·"true"␣  
    ···nginx.ingress.kubernetes.io/rewrite-target: ·/$2␣  
  ··hosts:␣  
    ···- ·host: ·microk8s␣  
      ····paths:␣  
        ·····- ·path: ·/containerapp(/|$) (.*)  
        ······pathType: ·Prefix
```

- Parameter für ingress.yaml in values.yaml definieren.

Helm-Chart erstellen - Templates überprüfen

```
~/beispiele/helm/containerapp$ helm template containerapp ./containerapp
---
# Source: containerapp/templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: containerapp
...
```

- Die durch helm erstellten Manifest-Dateien werden mit *helm template* auf der Standard-Ausgabe ausgegeben.

Agenda

- Zielsetzung
- Helm-Chart erstellen
- Helm-Chart testen
- Automatisches Deployment mit Helm

Helm-Chart testen - Manuelle Installation

```
$ helm install containerapp ./containerapp
NAME: containerapp
LAST DEPLOYED: Mon Mar 16 05:18:32 2026
NAMESPACE: dev-namespace
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
1. Get the application URL by running these commands:
  http://microk8s/containerapp(/|$) (.*)
```

```
$ helm list
...
$ helm status containerapp
...
```

- *helm install*: Installiert das Helm-Chart.
- *helm list*: Listet die installierten Helm-Charts des Namespaces auf.
- *helm status*: Zeigt den Installations-Status eines Helm-Charts an.

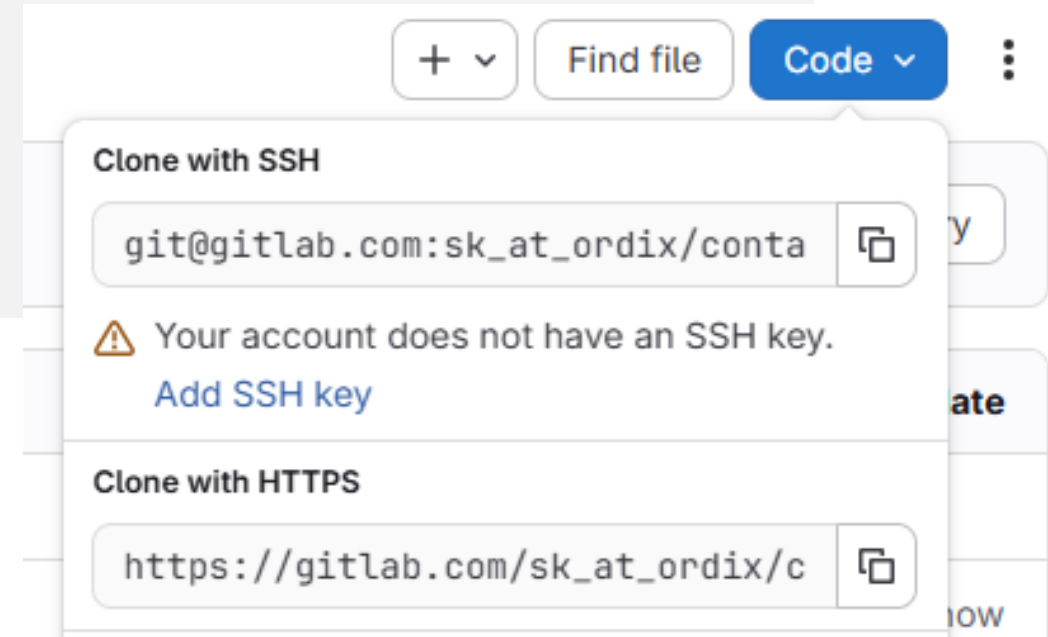
Agenda

- Zielsetzung
- Helm-Chart erstellen
- Helm-Chart testen
- Automatisches Deployment mit Helm

Automatisches Deployment - Helm-Chart in das Git-Repository (1)

```
$ mkdir git
$ cd git
```

```
$ git clone https://gitlab.com/sk_at_ordix/containerapp.git
$ cd containerapp
$ ls
pom.xml README.md src
$ git status
On branch main
Your branch is up to date with 'origin/main'.
```



- Projekt auschecken: git-URL ist über die Projekt-Homepage ermittelbar (s. rechts)
- *git clone*: Main-Branch des Remote-Repositories in den Workspace kopieren.
- *git status*: Zeigt Veränderungen im Workspace an.

Automatisches Deployment - Helm-Chart in das Git-Repository (2)

```
$ git checkout automated-kubernetes-manifest-deployment
...
$ git checkout -b automated-helm-deployment

$ rm -r k8s
$ git status
On branch automated-helm-deployment
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    k8s/deployment.yaml
        deleted:    k8s/ingress.yaml
        deleted:    k8s/service.yaml
no changes added to commit (use "git add" and/or "git commit -a")
$ git add -A
$ git commit -m 'loeschen der Manifest-Dateien'
```

- Neuen Branch auf Grundlage des Automatischen kubectl-Deployments anlegen.
- k8s-Verzeichnis mit Manifest-Dateien löschen.
- Stand des Branches committen.

Automatisches Deployment - Helm-Chart in das Git-Repository (3)

```
$ mkdir chart
$ cd chart
$ cp -r ~/beispiele/helm/containerapp/containerapp/* .
$ ls
charts Chart.yaml templates values.original.yaml values.yaml
$ git add -A
$ git commit -m 'helm-chart hinzufuegen'
$ git push --set-upstream origin automated-helm-deployment
```

- Die Dateien des Helm-Charts in den Workspace kopieren.
- Änderungen in den lokalen Branch committen.
- Lokalen Branch an das zentral Git-Repository pushen.
- Das Build-Skript (.gitlab-ci.yml) ist nicht angepasst: Der Build nach dem push schlägt fehl.

Automatisches Deployment - Helm-Chart in das Git-Repository (4)

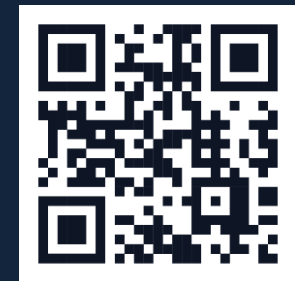
```
deploy:
  stage: deploy
  tags:
    - onkubernetes
  image:
    name: alpine/helm:3.14.4
    entrypoint: [""]
  before_script:
    - apk add --no-cache kubectl
  script:
    - kubectl config use-context "$KUBE_CONTEXT"
    - kubectl create namespace "$KUBE_NAMESPACE" --dry-run=client -o yaml | kubectl apply -f -
    - |
    - helm upgrade --install "$HELM_RELEASE" ./chart \
    - --namespace "$KUBE_NAMESPACE" \
    - --set image.repository="$IMAGE_NAME" \
    - --set image.tag="$IMAGE_TAG"
    - helm status "$HELM_RELEASE" -n "$KUBE_NAMESPACE"
```

- Image mit kubectl und helm-Integration aussuchen.
- Variable \$HELM_RELEASE = containerapp definieren

Automatisches Deployment - Helm-Chart in das Git-Repository (4)

- Vor dem Commit der Änderung sicher stellen, dass es keine Konflikte bei Ingress mit anderen Ingress-Instanzen gibt: Vorherige Installation deinstallieren (helm list, helm uninstall)
- Nach dem Commit:
 - Pipeline kontrollieren.
 - Anwendung testen.

**Vielen Dank für
Ihre Aufmerksamkeit**



ORDIX AG
Aktiengesellschaft für Softwareentwicklung,
Schulung, Beratung und Systemintegration

Seminarzentrum Wiesbaden
Kreuzberger Ring 13
65205 Wiesbaden
Tel.: 0611 77840-00
Fax: 0180 1 67349 0

seminare@ordix.de
<https://www.seminare.ordix.de/>